

ConfigMaps & Secrets

configuring applications

Kubernetes in Action · Chapter 7

minikube 4.5.2 (1 CP + 3 Worker) · Calico CNI · Podman

오늘 다룰 내용

#	주제	핵심
1	Command & Args	ENTRYPOINT/CMD 오버라이드
2	환경변수	env 하드코딩의 한계
3	ConfigMap	Pod과 설정 분리
4	configMap Volume	설정 파일 주입 + 자동 갱신
5	Secret	민감 정보 암호화 보호

6장이 "데이터 저장"이었다면, 7장은 *****설정 주입*****입니다

왜 설정 관리가 필요한가?

같은 이미지, 다른 환경

환경별 배포의 현실

같은 앱 이미지를 여러 환경에 배포해야 한다:

```
dev → DB: dev-db.internal,  DEBUG: true
prod → DB: prod-db.internal, DEBUG: false
```

설정을 주입하는 방법의 진화:

- | | |
|----------------------|---|
| [1] args 오버라이드 | → Pod YAML에 고정 |
| [2] env 하드코딩 | → 환경별 YAML 분리 필요 |
| [3] ConfigMap + env | → Pod와 설정 분리!  |
| [4] configMap Volume | → 설정 파일 전체 주입 + 자동 갱신 |
| [5] Secret | → 민감 정보 암호화 |

핵심: 같은 Pod YAML + 환경별 ConfigMap = 이식성 확보

1. Command & Arguments

Docker ENTRYPOINT/CMD → K8S command/args

Shell form vs Exec form

형식	PID 1	SIGTERM	판정
Exec ["node", "app.js"]	앱 프로세스	직접 수신	✅ 사용
Shell node app.js	/bin/sh	앱에 전달 안 됨	❌ 금지

Kubernetes에서 오버라이드:

Docker	Kubernetes	설명
ENTRYPOINT	command	실행 명령어
CMD	args	전달 인자

```
containers:
- image: luksa/fortune:args
  args: ["2"]          # CMD "10" → "2" 오버라이드
```

command 와 args 는 Pod 생성 후 변경 불가

Demo: args 오버라이드

```
$ kubectl apply -f fortune-pod-args.yaml
pod/fortune-args created

$ kubectl logs fortune-args -c html-generator | head -1
Configured to generate new fortune every 2 seconds
```

- 이미지의 기본 CMD는 "10" (10초)
- `args: ["2"]` 로 오버라이드 → 2초 간격 동작
- 이미지 재빌드 없이 동작 변경 가능

하지만 "2"라는 값이 Pod YAML에 고정되어 있는 것이 한계

2. 환경변수

env 하드코딩의 문제

환경변수 주입

```
containers:
- image: luksa/fortune:env
  env:
  - name: INTERVAL
    value: "5"           # Pod YAML에 값이 직접 들어감
```

문제: 환경별 YAML이 따로 필요

```
fortune-pod-dev.yaml  → INTERVAL: "5"
fortune-pod-prod.yaml → INTERVAL: "30"
```

변수 간 참조도 가능:

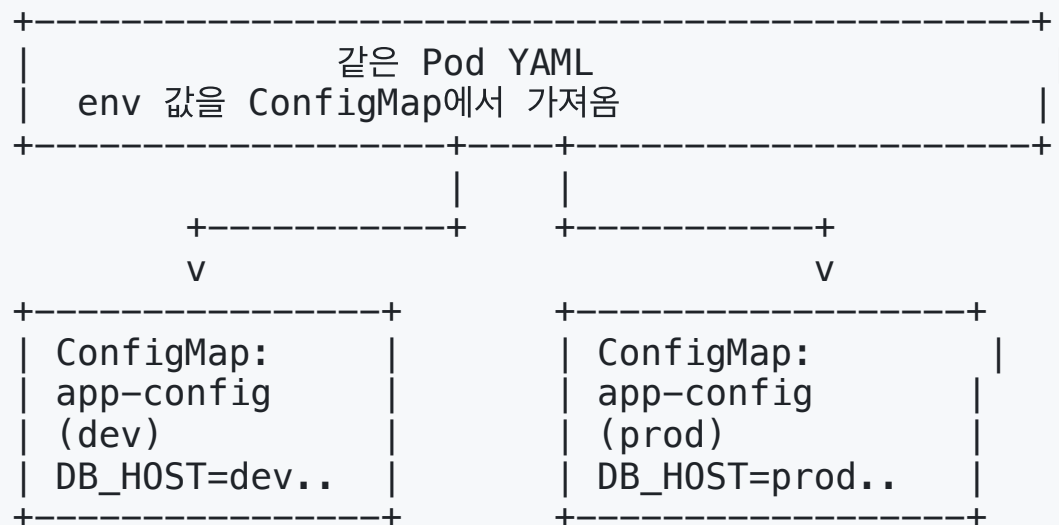
```
env:
- name: FIRST_VAR
  value: "foo"
- name: SECOND_VAR
  value: "${FIRST_VAR}bar"    # → "foobar"
```

환경변수 값이 Pod YAML에 하드코딩 → ConfigMap으로 분리하자

3. ConfigMap

Pod과 설정을 완전 분리

ConfigMap 개념



핵심: ConfigMap 이름만 같으면, 같은 Pod YAML을 재사용 가능

ConfigMap 생성 방법

```
# 리터럴
kubectl create configmap fortune-config \
  --from-literal=sleep-interval=25

# 파일에서 (파일명 = key)
kubectl create configmap my-config \
  --from-file=nginx.conf

# 디렉토리 전체
kubectl create configmap my-config \
  --from-file=config-dir/

# 조합
kubectl create configmap my-config \
  --from-file=foo.json \
  --from-literal=some=thing
```

ConfigMap key는 유효한 DNS subdomain (영숫자, 대시, 밑줄, 점)

ConfigMap → 환경변수 전달

```
env:  
- name: INTERVAL  
  valueFrom:  
    configMapKeyRef:  
      name: fortune-config      # ConfigMap 이름  
      key: sleep-interval      # key
```

envFrom으로 전체 전달:

```
envFrom:  
- prefix: CONFIG_                # 선택: 접두사  
  configMapRef:  
    name: my-config-map
```

ConfigMap key	환경변수	비고
FOO	CONFIG_FOO	✓
BAR	CONFIG_BAR	✓
FOO-BAR	⚠ 스킵	대시는 유효하지 않은 변수명

Demo: ConfigMap → env

```
$ kubectl create configmap fortune-config \
  --from-literal=sleep-interval=10
configmap/fortune-config created

$ kubectl apply -f fortune-pod-configmap-env.yaml

$ kubectl logs fortune-configmap-env -c html-generator | head -1
Configured to generate new fortune every 10 seconds
```

- Pod YAML에 "10"이라는 값이 직접 들어가지 않음
- ConfigMap만 교체하면 환경별 배포 가능!

하지만 env로 주입한 값은 Pod 재시작 전까지 변경 불가

4. configMap Volume

설정 파일 주입 + 자동 갱신

configMap Volume 개념

ConfigMap entry를 파일로 컨테이너에 노출:

```
volumes:  
- name: config  
  configMap:  
    name: fortune-config  
    items:  
    - key: my-nginx-config.conf  
      path: my-nginx-config.conf
```

결과:

```
/etc/nginx/conf.d/  
└─ my-nginx-config.conf ← ConfigMap entry의 내용
```

자동 갱신 메커니즘 (symlink atomic swap):

```
..data -> ..4984_09_04_12_15_06 ← symlink  
my-nginx-config.conf -> ..data/my-nginx-config.conf
```


subPath — 개별 파일 마운트

문제: 디렉토리 마운트 시 기존 파일이 숨겨짐

```
# subPath로 해결 (기존 파일 유지)
volumeMounts:
- name: config
  mountPath: /etc/someconfig.conf # 파일 경로
  subPath: myconfig.conf         # Volume의 특정 파일만
```

subPath의 대가:

방식	기존 파일	자동 갱신
디렉토리 마운트	숨겨짐 ✗	✓
subPath	유지됨 ✓	✗

Demo: Nginx gzip 활성화 (configMap Volume)

```
$ kubectl apply -f fortune-pod-configmap-volume.yaml  
  
$ kubectl exec fortune-configmap-volume -c web-server \  
  -- ls /etc/nginx/conf.d  
my-nginx-config.conf  
  
$ curl -H "Accept-Encoding: gzip" -I localhost:8080  
HTTP/1.1 200 OK  
Content-Encoding: gzip           ← gzip 활성화!
```

Demo: ConfigMap 수정 → 자동 갱신 (재시작 없이!)

```
# gzip off로 변경
$ kubectl patch configmap fortune-config --type merge \
  -p '{"data":{"my-nginx-config.conf":"server { gzip off; ... }"}}'

# ~30초 후 자동 갱신 확인
$ kubectl exec ... -- cat /etc/nginx/conf.d/my-nginx-config.conf | grep gzip
gzip off;                                ← 자동 갱신됨!

# Nginx reload 후 검증
$ kubectl exec ... -- nginx -s reload
$ curl -H "Accept-Encoding: gzip" -I localhost:8888
HTTP/1.1 200 OK
(Content-Encoding 없음)                ← gzip 비활성화!
```

Pod 재시작 없이 ConfigMap 수정만으로 설정 변경 완료!

5. Secret

민감 정보 암호화 보호

Secret vs ConfigMap

항목	ConfigMap	Secret
Node 저장	디스크	tmpfs (RAM)
etcd 저장	평문	암호화
YAML 표시	평문	Base64
노드 배포	전체	필요 노드만
용도	일반 설정	민감 정보

선택 기준:

- 일반 설정 → ConfigMap
- 비밀번호, 인증서, API 키, 토큰 → **Secret**

default-token Secret

모든 Pod에 자동 마운트:

```
/var/run/secrets/kubernetes.io/serviceaccount/  
├── ca.crt      ← API Server CA 인증서  
├── namespace   ← Pod의 Namespace  
└── token       ← ServiceAccount JWT
```

비활성화: `automountServiceAccountToken: false`

Secret 생성:

```
# generic Secret  
kubectl create secret generic fortune-https \  
    --from-file=https.key --from-file=https.cert  
  
# TLS Secret (Ingress용)  
kubectl create secret tls my-tls \  
    --cert=cert.pem --key=key.pem
```

Demo: Secret으로 HTTPS 활성화

```
# TLS 인증서 생성 + Secret 등록
$ openssl genrsa -out https.key 2048
$ openssl req -new -x509 -key https.key -out https.cert ...
$ kubectl create secret generic fortune-https \
  --from-file=https.key --from-file=https.cert
```

```
# Pod에서 Secret Volume 마운트
volumes:
- name: certs
  secret:
    secretName: fortune-https
```

```
# tmpfs 확인 - 디스크에 기록 안 됨!
$ kubectl exec fortune-https -c web-server -- mount | grep certs
tmpfs on /etc/nginx/certs type tmpfs (ro,relatime)
```

```
# HTTPS 서빙 성공
$ curl -sk https://localhost:8443
Give him an evasive answer.
```

Secret 보안 주의사항

환경변수 vs Volume:

방식	보안 수준	이유
Volume 마운트	✓ 높음	tmpfs, 파일 권한 제어
환경변수	⚠ 낮음	/proc/<pid>/environ 노출 가능

운영 권장: 민감 정보는 가능하면 Volume 마운트로 전달

stringData — Base64 없이 직접 입력 (write-only):

```
kind: Secret
stringData:
  api-key: my-plain-text-key    # YAML에는 평문
data:
  password: cGFzc3dvcmQ=      # 기존 방식 (Base64)
```


전체 요약

설정 주입 방법 비교

7장 요약 — 설정 주입의 진화

방법	장점	자동 갱신	용도
args	단순	✗	인자 1~2개
env (고정)	단순	✗	프로토타입
env + ConfigMap	분리됨	✗	일반 설정
configMap Volume	파일 주입	✓ ~30s	설정 파일
Secret Volume	암호화, tmpfs	✓	민감 정보

운영 패턴:

```
env:
- name: DB_HOST      # 일반 → ConfigMap
  valueFrom: { configMapKeyRef: ... }
- name: DB_PASSWORD  # 민감 → Secret
  valueFrom: { secretKeyRef: ... }
volumeMounts:
- name: config        # 설정 파일 → configMap Volume
- name: certs          # 인증서 → Secret Volume (tmpfs)
```

한 문장 핵심

앱 설정은 ConfigMap으로 분리하고, 민감 정보는 Secret으로 보호하라.

같은 Pod YAML + 환경별 ConfigMap/Secret = 이식성 확보.

Q&A

질문	답변
ConfigMap 수정 시 env도 바뀌나?	❌ Pod 재시작 필요. Volume만 자동 갱신
subPath 자동 갱신 안 되는 이유?	symlink가 아닌 직접 바인딩이라 atomic swap 불가
Secret env는 안전한가?	/proc 노출 가능. Volume 마운트 선호
최대 크기?	1MB (Secret은 Base64로 실질 ~750KB)